

# C++ Functions

Modular Logic & Code Reusability

Efficiency through Decomposition

---

**Core Programming Concepts**

# 1. Defining a Function

---

A function is an independent, named block of code designed to perform a specific task. Think of a function as a "Mini-Program" that sits inside your main program, waiting to be used.

## Key Philosophies:

- **Don't Repeat Yourself (DRY):** Write code once, use it everywhere.
- **Abstraction:** You don't need to know how `sqrt()` works to find a square root; you just need to know its name.

## The Real-Life Analogy

Think of a **Coffee Machine** as a function.

- **Input:** Water and Coffee beans (Parameters)
- **Action:** Brewing internally (Function Body)
- **Output:** A hot cup of Coffee (Return Value)

## 2. Structure & Categories

---

Every function in C++ requires a specific signature to tell the compiler how to handle it.

```
return_type functionName(parameter_list) { // Body of the function return
value; // (unless type is void) }
```

### Types of Functions:

1. **Built-in (Predefined):** Functions provided by C++ standard libraries like `main()`, `sqrt()`, or `pow()`.
2. **User-Defined:** Functions you create to solve your specific logic problems.

### Common Return Types:

| Type | Example Return                      |
|------|-------------------------------------|
| void | No return (just performs an action) |
| int  | Whole numbers (e.g., 42)            |
| bool | Logic results (true/false)          |

## 3. Declaration vs. Definition

---

In C++, order matters. A function must be known by the compiler before it is called. You can handle this in two ways:

### Approach A: Definition before Main

```
void hello() { cout << "Hi!"; } int main() { hello(); return 0; }
```

### Approach B: Prototype (Declaration) First

This is the professional standard. You announce the function at the top and specify the details at the bottom.

```
void hello(); // Prototype int main() { hello(); } void hello() { cout << "Hi!"; }
```

## 4. Passing Data: Parameters

---

Functions become truly powerful when you can feed them data. **Parameters** act as placeholders inside the function, while **Arguments** are the actual values you pass into them.

```
void greet(string name) { // 'name' is the parameter cout << "Welcome " <<
name; } int main() { greet("Neo"); // "Neo" is the argument }
```

### Multiple Parameters:

You can pass multiple items by separating them with commas. The order must match exactly.

```
void user(string name, int age) { cout << name << " is " << age << " years
old."; }
```

## 5. Returning Data

---

Often, you don't want a function to print something; you want it to **calculate** something and give the result back to the main program.

```
int multiply(int a, int b) { return a * b; // Sends the result back } int  
main() { int result = multiply(10, 5); // result becomes 50 cout <<  
result; }
```

**Important:** Once a return statement is reached, the function stops immediately. Any code written after it inside the function will never execute.

## 6. Memory Logic: Value vs Reference

---

How C++ handles the memory of your arguments is crucial for performance and correctness.

### 1. Pass by Value (The Default)

C++ makes a **copy**. The original variable is safe and remains unchanged.

### 2. Pass by Reference

C++ sends the **original address** using the & operator. If the function changes the value, the original variable in `main()` changes too.

```
void update(int &x) { x = 100; } int main() { int score = 10; update(score);  
cout << score; // Outputs 100! }
```

## 7. Function Overloading

---

In C++, multiple functions can have the **same name** as long as they have **different parameters**. This is known as Overloading.

```
int add(int x, int y) { return x + y; } double add(double x, double y)
{ return x + y; }
```

When you call `add()`, the compiler looks at the data types you provided and automatically picks the correct version. This makes your API much cleaner for users.

Think of a "Power" button. On a TV, it turns on the screen. On a Microwave, it starts the heater. Same name ("Power"), different context (Parameter/Object).



## 8. Best Practices & Pitfalls

---

### Common Mistakes:

- **Mismatched Types:** Returning a string in a function declared as `int`.
- **Unreachable Code:** Writing logic after a return statement.
- **Narrow Scope:** Trying to use a variable inside `main` that was declared inside a function.

### Pro-Tips for Clean Code:

- **Descriptive Names:** Use verbs. `calculateTax()` is better than `tax()`.
- **One Task:** A function should do one thing perfectly. If a function is doing three things, split it into three functions.
- **Keep it Small:** If a function is longer than 20-30 lines, it might be too complex.

## 9. Function Lab: Skill Challenges

---

Put your knowledge into practice with these three challenges:

### Challenge 1: The Mathematician

Create a function `getMax(int a, int b)` that returns the larger of two numbers. Test it with user input in `main()`.

### Challenge 2: Prime Check

Write a function `bool isPrime(int n)` that returns true if a number is prime and false otherwise. Use a loop inside the function.

### Challenge 3: The Reformatter

Write a function that takes a string `name` and an `int id` and prints them in a formatted employee badge style.

## END OF C++ FUNCTIONS MODULE

Next steps: Classes and Objects

MASTERED